Introduzione al linguaggio SQL

Giorgio Bar – giorgio.bar@to.infn.it

Scuola di Specializzazione in Fisica Medica – Università di Torino

Anno accademico 2021/2022 (anno solare 2022/2023)

Il linguaggio SQL

- Linguaggio di riferimento per le basi di dati relazionali
  - Structured Query Language

- Un linguaggio a livello di set
  - Gli operatori operano su relazioni
  - Il risultato è sempre una relazione

- Un linguaggio dichiarativo
  - Si specifica l'obiettivo e non il modo in cui ottenerlo, ad un livello di astrazione superiore rispetto ai linguaggi di programmazione procedurali come l'algebra relazionale.

Istruzioni del linguaggio

Il linguaggio SQL può essere suddiviso in

- DDL (Data Definition Language)
  - Definire lo schema di una base di dati relazionale
  - Definire i privilegi di accesso degli utenti
  - Gestire le transazioni (una sequenza di operazioni di modifica che porta la base dati da uno stato consistente a un altro stato consistente, ”tutto-o-niente”)
- DML (Data Manipulation Language)
  - Istruzioni per leggere e scrivere i dati

Data Definition Language

- Definizione dello schema di una base di dati. Creazione, modifica e cancellazione di tabelle e viste (tabelle virtuali il cui contenuto è ottenuto a partire da altre tabelle)
  - CREATE, ALTER, DROP TABLE / VIEW
- Definizione di strutture dati accessorie per recuperare in modo efficiente i dati. Creazione e cancellazione di indici
  - CREATE, DROP INDEX
- Concessione e revoca dei privilegi di accesso alle risorse
  - GRANT, REVOKE
- Definizione di transazioni
  - COMMIT, ROLLBACK

Data Manipulation Language

- Interrogazione della base di dati per estrarre i dati di interesse
  - SELECT
- Inserimento di nuove informazioni
  - INSERT
- Aggiornamento di dati
  - UPDATE
- Cancellazione di dati
  - DELETE

Istruzione SELECT: fondamenti

Struttura dell'istruzione SELECT

SELECT [DISTINCT] *elenco_attributi_da_visualizzare*

FROM *elenco_tabelle_da_utilizzare*

[WHERE *condizioni_di_tupla*]

[GROUP BY *attributi_di_raggruppamento*]

[HAVING *condizioni_su_aggregati*]

[ORDER BY *attributi_di_ordinamento*];

Base dati di esempio

- Tabella P: prodotti

  P (codP, NomeP, Colore, Peso, Magazzino)

- Tabella F: fornitori

  F (codF, NomeF, Rating, Sede)

- Tabella FP: forniture, mette in relazione i prodotti con i fornitori che li forniscono

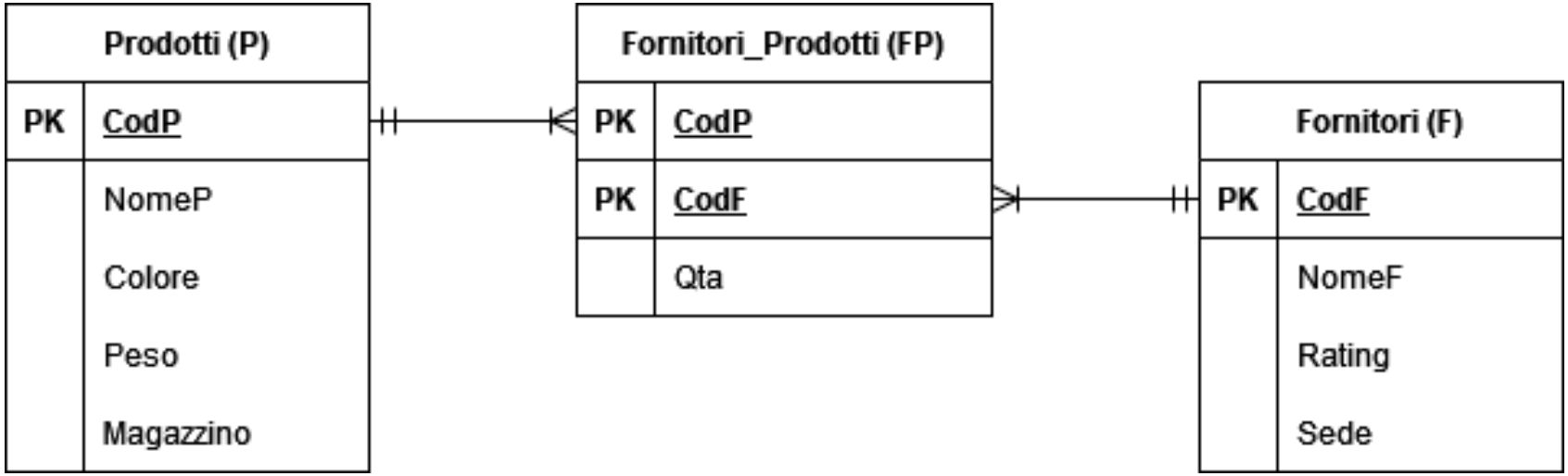
  FP (codF, codP, Qta)

Base dati di esempio

P (codP, NomeP, Colore, Peso, Magazzino)

F (codF, NomeF, Rating, Sede)

FP (codF, codP, Qta)

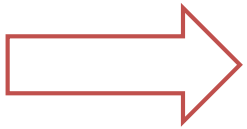


SELECT base

- Trovare il codice di tutti i prodotti

```
SELECT CodP
FROM P;
```

CodP	NomeP	Colore	Peso	Magazzino
P1	Trion	Rosso	40	Torino
P2	Speed	Giallo	48	Milano
P3	Airtech	Blu	48	Roma
P4	Airtech	Blu	44	Torino
P5	Futura	Blu	40	Milano
P6	Zen	Rosso	42	Torino



CodP
P1
P2
P3
P4
P5
P6

SELECT base

- Trovare il codice dei prodotti forniti da almeno un fornitore

CodF	CodP	Qta
F1	P1	300
F1	P2	200
F1	P3	400
F1	P4	200
F1	P5	100
F1	P6	100
F2	P1	300
F2	P2	400
F3	P2	200
F4	P3	200
F4	P4	300
F4	P5	400

```
SELECT CodP
FROM FP;
```

CodP
P1
P2
P3
P4
P5
P6
P1
P2
P2
P3
P4
P5

Non effettua la rimozione dei duplicati (a differenza della proiezione in algebra relazionale).

SELECT DISTINCT

- Trovare il codice dei prodotti diversi forniti da almeno un fornitore

CodF	CodP	Qta
F1	P1	300
F1	P2	200
F1	P3	400
F1	P4	200
F1	P5	100
F1	P6	100
F2	P1	300
F2	P2	400
F3	P2	200
F4	P3	200
F4	P4	300
F4	P5	400

```
SELECT DISTINCT CodP
FROM FP;
```

CodP
P1
P2
P3
P4
P5
P6

Eliminazione dei duplicati

Struttura dell'istruzione SELECT

SELECT [DISTINCT] *elenco_attributi_da_visualizzare*

FROM *elenco_tabelle_da_utilizzare*

[WHERE *condizioni_di_tupla*]

[GROUP BY *attributi_di_raggruppamento*]

[HAVING *condizioni_su_aggregati*]

[ORDER BY *attributi_di_ordinamento*];

Clausola WHERE

- Permette di esprimere condizioni di selezione applicate singolarmente ad ogni tupla
- Espressione booleana di predicati
- Predicati semplici
  - Confronto tra attributi e costanti
  - Gestione dei valori NULL

Operator	Description
<	less than
>	greater than
<=	less than or equal to
>=	greater than or equal to
=	equal
<> or !=	not equal

https://www.postgresql.org/docs/10/functions-comparison.html

Clausola WHERE

- Trovare il codice dei fornitori di Milano

```
SELECT CodF
FROM F
WHERE Sede = 'Milano';
```

CodF	NomeF	Rating	Sede
F1	Atlante	2	Torino
F2	Oceano	1	Milano
F3	Crono	3	Milano
F4	Prometeo	2	Torino
F5	Ceo	3	Venezia

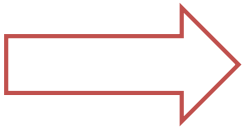
CodF
F2
F3

Clausola WHERE

- Trovare il codice e il rating dei fornitori che non hanno sede a Milano

```
SELECT CodF, Rating
FROM F
WHERE Sede <> 'Milano';
```

CodF	NomeF	Rating	Sede
F1	Atlante	2	Torino
F2	Oceano	1	Milano
F3	Crono	3	Milano
F4	Prometeo	2	Torino
F5	Ceo	3	Venezia



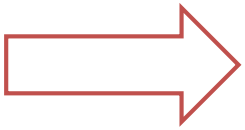
CodF	Rating
F1	2
F4	2
F5	3

Espressioni booleane

- Trovare il codice dei fornitori di Milano con un rating superiore a 2

```
SELECT CodF
FROM F
WHERE Sede = 'Milano' AND Rating > 2;
```

CodF	NomeF	Rating	Sede
F1	Atlante	2	Torino
F2	Oceano	1	Milano
F3	Crono	3	Milano
F4	Prometeo	2	Torino
F5	Ceo	3	Venezia



CodF
F3

Espressioni booleane

- Trovare il codice e il rating dei fornitori di Milano o di Torino

```
SELECT CodF, Rating
FROM F
WHERE Sede = 'Milano' OR Sede = 'Torino';
```

CodF	NomeF	Rating	Sede
F1	Atlante	2	Torino
F2	Oceano	1	Milano
F3	Crono	3	Milano
F4	Prometeo	2	Torino
F5	Ceo	3	Venezia

CodF	Rating
F1	2
F2	1
F3	3
F4	2

Gestione dei valori NULL

- Trovare il codice e il nome dei prodotti con peso maggiore di 44

```
SELECT CodP, NomeP
FROM P
WHERE Peso > 44;
```

CodP	NomeP	Colore	Peso	Magazzino
P1	Trion	Rosso	40	Torino
P2	Speed	Giallo	48	Milano
P3	Airtech	Blu	48	Roma
P4	Airtech	Blu	44	Torino
P5	Futura	Blu	NULL	Milano
P6	Zen	Rosso	42	Torino

CodP	NomeP
P2	Speed
P3	Airtech

Gestione dei valori NULL

- Le tuple per cui il peso è NULL non sono selezionate
- In presenza di un valore NULL qualsiasi predicato di confronto è falso.
- Operatore speciale IS

  attributo IS [NOT] NULL

- Trovare il codice e il nome dei prodotti con peso maggiore di 44 o il cui peso non è indicato

```
SELECT CodP, NomeP
FROM P
WHERE Peso > 44 OR Peso IS NULL;
```

Struttura dell'istruzione SELECT

SELECT [DISTINCT] *elenco_attributi_da_visualizzare*

FROM *elenco_tabelle_da_utilizzare*

[WHERE *condizioni_di_tupla*]

[GROUP BY *attributi_di_raggruppamento*]

[HAVING *condizioni_su_aggregati*]

[ORDER BY *attributi_di_ordinamento*];

Ordinamento del risultato

- Clausola ORDER BY

  ORDER BY nome_attributo [ASC | DESC]
  {, nome_attributo [ASC | DESC]}

- L'ordinamento implicito è crescente (ASC)
- Gli attributi di ordinamento devono comparire nella clausola SELECT
  - anche implicitamente (come SELECT *)

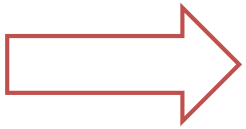
https://www.postgresql.org/docs/13/queries-order.html

Ordinamento del risultato

- Trovare il codice dei prodotti e il loro peso, ordinando il risultato in ordine decrescente di peso

```
SELECT CodP, Peso
FROM P
ORDER BY Peso DESC;
```

CodP	NomeP	Colore	Peso	Magazzino
P1	Trion	Rosso	40	Torino
P2	Speed	Giallo	48	Milano
P3	Airtech	Blu	48	Roma
P4	Airtech	Blu	44	Torino
P5	Futura	Blu	40	Milano
P6	Zen	Rosso	42	Torino



CodP	Peso
P2	48
P3	48
P4	44
P6	42
P1	40
P5	40

Ordinamento del risultato

- Trovare tutte le informazioni sui prodotti, ordinando il risultato in ordine crescente di nome e decrescente di peso

```
SELECT CodP, NomeP, Colore, Peso, Magazzino
FROM P
ORDER BY NomeP, Peso DESC;

SELECT *
FROM P
ORDER BY NomeP, Peso DESC;
```

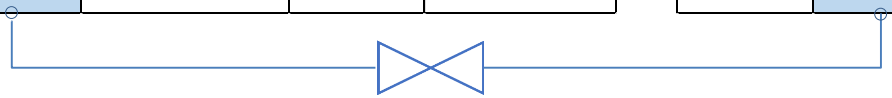
Join

- Se l'argomento della clausola FROM è costituito da più di una tabella, le condizioni espresse dalla clausola WHERE vengono applicate sul prodotto cartesiano delle tabelle elencate.
- Un join (interno) può essere specificato indicando in modo esplicito le condizioni che esprimono il legame tra le tabelle.

Join

CodF	NomeF	Rating	Sede
F1	Atlante	2	Torino
F2	Oceano	1	Milano
F3	Crono	3	Milano

CodP	CodF	Qta
P1	F2	300
P2	F2	400
P2	F3	200

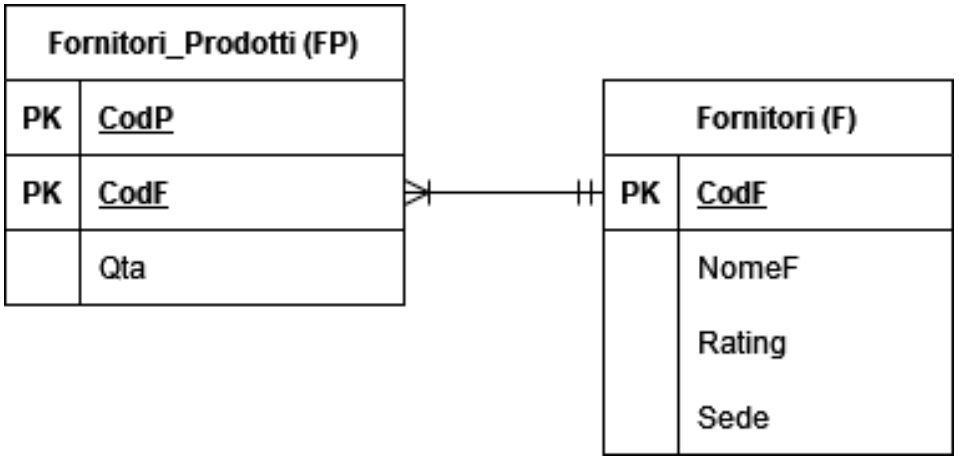


CodF	NomeF	Rating	Sede	CodP	Qta
F2	Oceano	1	Milano	P1	300
F2	Oceano	1	Milano	P2	400
F3	Crono	3	Milano	P2	200

```
SELECT F.CodF, NomeF, Rating, Sede, CodP, Qta
FROM F, FP
WHERE F.CodF = FP.CodF;
```

Join

- Trovare il nome dei fornitori che forniscono il prodotto P2




```
SELECT NomeF
FROM F, FP
WHERE F.CodF = FP.CodF
AND CodP = 'P2;
```

prodotto cartesiano

condizione di join

Join

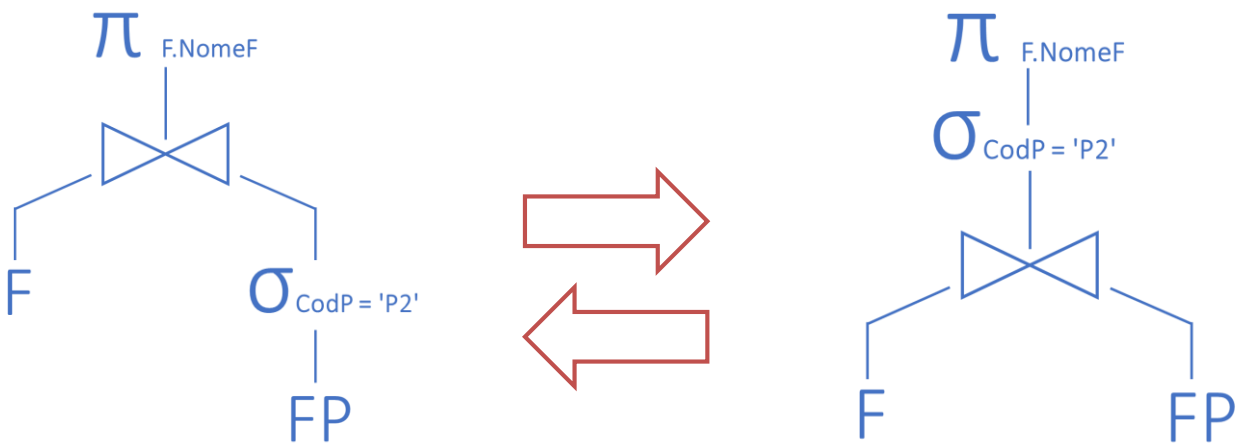
- Trovare il nome dei fornitori che forniscono il prodotto P2

```
SELECT NomeF                          DISTINCT ?
FROM F, FP          prodotto cartesiano
WHERE F.CodF = FP.CodF        condizione di join
AND CodP = 'P2;
```

F.CodF	F.NomeF	F.Rating	F.Sede	FP.CodF	FP.CodP	FP.Qta
F1	Atlante	2	Torino	F1	P2	200
F2	Oceano	1	Milano	F2	P2	400
F3	Crono	3	Milano	F3	P2	200

Join

- Trovare il nome dei fornitori che forniscono il prodotto P2
  - In algebra relazionale



Join

- Dichiaratività del linguaggio SQL
  - In algebra relazionale si definisce l'ordine in cui sono applicati gli operatori
  - In SQL l'ordine migliore è scelto dall'ottimizzatore
  - Il risultato e l'efficienza sono indipendenti dall'ordine delle tabelle nella clausola FROM
  - Il risultato e l'efficienza sono indipendenti dall'ordine dei predicati nella clausola WHERE

Join

- Trovare il nome dei fornitori che forniscono almeno un prodotto rosso

```
SELECT NomeF                                DISTINCT ?
FROM F, FP, P
WHERE F.CodF = FP.CodF
      AND P.CodP = FP.CodP
      AND Colore = 'Rosso';
```

- Clausola FROM con N tabelle
  - Almeno N-1 condizioni di join nella clausola WHERE

Join

- Trovare le coppie di codici dei fornitori tali che entrambi abbiano sede nella stessa città

```
SELECT F1.CodF, F2.CodF
FROM F AS F1, F AS F2
WHERE F1.Sede = F2.Sede;
```

F1

CodF	NomeF	Rating	Sede
F1	Atlante	2	Torino
F2	Oceano	1	Milano
F3	Crono	3	Milano
F4	Prometeo	2	Torino
F5	Ceo	3	Venezia

F2

CodF	NomeF	Rating	Sede
F1	Atlante	2	Torino
F2	Oceano	1	Milano
F3	Crono	3	Milano
F4	Prometeo	2	Torino
F5	Ceo	3	Venezia

Join

- Trovare le coppie di codici dei fornitori tali che entrambi abbiano sede nella stessa città

```
SELECT F1.CodF, F2.CodF
FROM F AS F1, F AS F2
WHERE F1.Sede = F2.Sede;
```

F1.CodF	F2.CodF
F1	F1
F1	F4
F2	F2
F2	F3
F3	F2
F3	F3
F4	F1
F4	F4
F5	F5

- Sono presenti
  - Coppie di valori uguali
  - Permutazioni della stessa coppia di valori

Join

- Trovare le coppie di codici dei fornitori tali che entrambi abbiano sede nella stessa città

```
SELECT F1.CodF, F2.CodF
FROM F AS F1, F AS F2
WHERE F1.Sede = F2.Sede
      AND F1.CodF <> F2.CodF;
```

- Elimina le coppie di valori uguali

F1.CodF	F2.CodF
~~F1~~	~~F1~~
F1	F4
~~F2~~	~~F2~~
F2	F3
F3	F2
~~F3~~	~~F3~~
F4	F1
~~F4~~	~~F4~~
~~F5~~	~~F5~~

Join

- Trovare le coppie di codici dei fornitori tali che entrambi abbiano sede nella stessa città

```
SELECT F1.CodF, F2.CodF
FROM F AS F1, F AS F2
WHERE F1.Sede = F2.Sede
      AND F1.CodF < F2.CodF;
```

- Elimina anche le permutazioni della stessa coppia di valori

F1.CodF	F2.CodF
F1	F4
F2	F3

Join: sintassi alternativa

- Permette di specificare ulteriori tipi di join
  - LEFT [OUTER], RIGHT [OUTER], FULL [OUTER]
- Permette di distinguere
  - Condizioni di join
  - Condizioni di selezione sulle tuple

SELECT [DISTINCT] a*ttributi_da_visualizzare*

FROM *tabella* < INNER | LEFT [OUTER] | RIGHT [OUTER] | FULL [OUTER] > JOIN *tabella* ON *condizione_di_join*

[WHERE *condizioni_di_tupla*];

Join: sintassi alternativa

SELECT [DISTINCT] a*ttributi_da_visualizzare*

FROM *tabella* < INNER | LEFT [OUTER] | RIGHT [OUTER] | FULL [OUTER] > JOIN *tabella* ON *condizione_di_join*

[WHERE *condizioni_di_tupla*];

Una query LEFT [OUTER] JOIN fra le tabelle A e B include tutte le tuple di della tabella A (left) e solo le tuple di B che soddisfano la condizione di join.

Se per una tupla di A non ci sono tuple di B che soddisfano la condizione, nella tupla risultante le colonne di B assumeranno il valore NULL.

INNER Join

- Trovare il nome dei fornitori che forniscono almeno un prodotto rosso

```
SELECT NomeF
FROM F, FP, P
WHERE F.CodF = FP.CodF
      AND P.CodP = FP.CodP
      AND Colore = 'Rosso';
```

```
SELECT NomeF
FROM P INNER JOIN FP ON P.CodP = FP.CodP
     INNER JOIN F ON F.CodF = FP.CodF
WHERE P.Colore = 'Rosso';
```

OUTER Join

- Trovare il codice e il nome dei fornitori, insieme al codice dei relativi prodotti forniti, riportando anche i fornitori che non hanno forniture

```
SELECT F.CodF, NomeF, CodP
FROM F LEFT OUTER JOIN FP
ON F.CodF = FP.CodF;
```

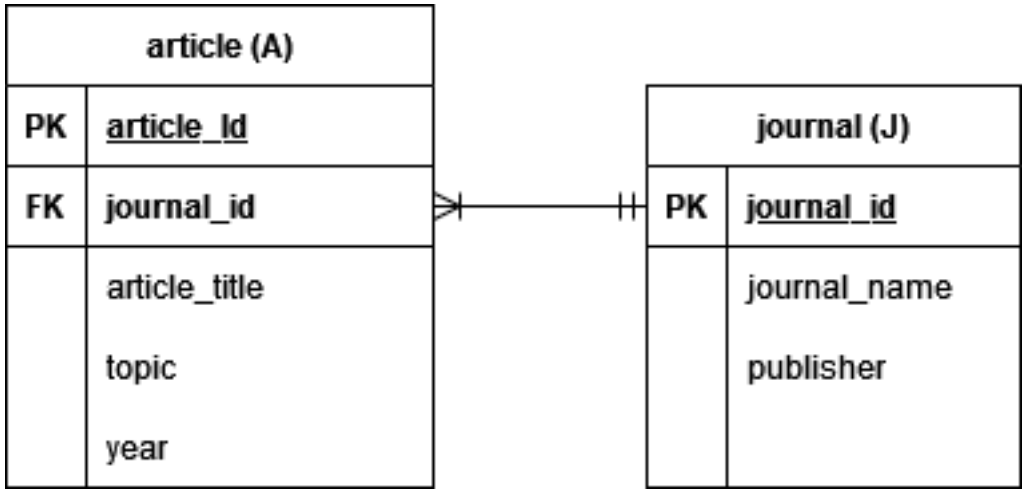
F.CodF	F.NomeF	FP.CodP
F1	Atlante	P1
…	…	…
F1	Atlante	P6
F2	Oceano	P1
F2	Oceano	P2
F3	Crono	P2
F4	Prometeo	P3
F4	Prometeo	P4
F4	Prometeo	P5
F5	Ceo	NULL

Esercizi

Dato lo schema logico relazionale costituito dalle seguenti tabelle (la chiave primaria è sottolineata e gli attributi opzionali sono indicati con un asterisco)

journal(<u>journal_id</u>, journal_name, publisher)

article(<u>article_id</u>, article_title, year*,
journal_id, topic)



Esercizi

Dato lo schema logico relazionale costituito dalle seguenti tabelle (la chiave primaria è sottolineata e gli attributi opzionali sono indicati con un asterisco)

journal(<u>journal_id</u>, journal_name, publisher)

article(<u>article_id</u>, article_title, year*,
journal_id, topic)

Esprimere in linguaggio SQL le seguenti interrogazioni:

1. Trovare il codice e il titolo delle riviste che hanno pubblicato **almeno un** articolo di argomento ‘NeuralNetworks’.
2. Trovare il codice e il titolo delle riviste che hanno pubblicato articoli di argomento 'NeuralNetworks‘ **oppure** 'DataMining'.

Soluzioni

```
journal( journal_id, journal_name, publisher)
article( article_id, article_title, year*,
         journal_id, topic)
```

1. Trovare il codice e il titolo delle riviste che hanno pubblicato **almeno un** articolo di argomento 'NeuralNetworks'.

```
-- soluzione #1
SELECT J.journal_id, J.journal_name
FROM journal J, article A
WHERE J.journal_id = A.journal_id
AND topic = 'NeuralNetworks';
```

Soluzioni

Trovare il codice e il titolo delle riviste che hanno pubblicato **almeno un** articolo di argomento 'NeuralNetworks'.

```
SELECT J.journal_id, J.journal_name
FROM journal J, article A
WHERE J.journal_id = A.journal_id
AND topic = 'NeuralNetworks';
```

```
+------------+---------------------------+
| journal_id | journal_name              |
+------------+---------------------------+
|       1001 | Neurocomputing            |
|       1002 | IEEE Access               |
|       1001 | Neurocomputing            |
|       1001 | Neurocomputing            |
|       1005 | Neural Processing Letters |
|       1007 | BMC Bioinformatics        |
+------------+---------------------------+
6 rows in set (0.00 sec)
```

Soluzioni

Trovare il codice e il titolo delle riviste che hanno pubblicato **almeno un** articolo di argomento 'NeuralNetworks'.

In algebra relazionale la proiezione elimina automaticamente i duplicati, la SELECT di SQL no.

```
SELECT DISTINCT J.journal_id, J.journal_name
FROM journal J, article A
WHERE J.journal_id = A.journal_id
AND topic = 'NeuralNetworks';
```

Soluzioni

```
journal( journal_id, journal_name, publisher)
article( article_id, article_title, year*,
         journal_id, topic)
```

2. Trovare il codice e il titolo delle riviste che hanno pubblicato articoli di argomento 'NeuralNetworks' **oppure** 'DataMining' .

```
SELECT J.journal_id, J.journal_name
FROM journal J, article A
WHERE J.journal_id = A.journal_id
AND (topic = 'NeuralNetworks' OR
     topic = 'DataMining');
```